

COMPUTER PROGRAMMING LAB REPORT

Lab report 12

Title: Pointer Basics and Pointer Arithmetic in C++

1. OBJECTIVE

- To understand the concept of pointers in C++
- To learn how pointers store and access memory addresses
- To practice dereferencing pointers
- To understand pointer arithmetic with arrays

2. THEORY

Pointers are special variables in C++ that store the **memory address** of another variable. Instead of holding a direct value, a pointer “points” to the location where data is stored.

Using pointers, we can:

- Access variable values indirectly (dereferencing)
- Work efficiently with arrays and memory
- Perform pointer arithmetic to navigate through array elements

Pointer arithmetic allows moving from one memory location to another (e.g., next array element) using operations like `ptr + 1`, `ptr + 2`, etc.

3. SOFTWARE USED

- DEV-C++ IDE
- GNU C++ Compiler

4. PROGRAMS

Program 1 — Pointer with Single Variable

```
#include <iostream>           // header file for input/output operations
using namespace std;         // standard namespace usage
int main() {                  // main function starts execution
    int num = 25;             // declaring and initializing integer variable

    int *ptr;                 // declaring pointer variable to store address of
    integer
```

```

ptr = &num;                // storing address of num inside pointer
cout << "Value of num: " << num << endl;
// displaying value directly
cout << "Value using pointer (*ptr): " << *ptr << endl;
// displaying value using pointer dereference
return 0;
}

```

Output

Value of num: 25
Value using pointer (*ptr): 25

Program 2 — Pointer Arithmetic with Array

```

#include <iostream>          // input/output library
using namespace std;        // standard namespace
int main() {                // main function starts
int numbers[] = {10, 20, 30, 40, 50};
// declaring and initializing integer array
int *ptr = numbers;         // pointer pointing to first element of array
cout << "Array elements using pointer arithmetic:" << endl;
// heading output
cout << "1st element: " << *ptr << endl;
// accessing first element
cout << "2nd element: " << *(ptr + 1) << endl;
// accessing second element using pointer arithmetic
cout << "3rd element: " << *(ptr + 2) << endl;   cout << "4th
element: " << *(ptr + 3) << endl;
    cout << "5th element: " << *(ptr + 4) << endl;
// accessing fifth element using pointer arithmetic

    return 0;                // program ends successfully
}

```

Output

Array elements using pointer arithmetic:
1st element: 10
2nd element: 20
3rd element: 30
4th element: 40
5th element: 50

5. CONCLUSION

In this lab, we studied pointers in C++ and learned how they store memory addresses of variables. We practiced dereferencing pointers to access values and also learned pointer arithmetic to access array elements efficiently. This lab helped strengthen understanding of memory handling and low-level data access in C++ programming.